

ASSIGNMENT-15.1

HT No:2303A51759

Batch No:28

Course:AI Assisted Coding

Task 1 :Student Records API

Task:

Use AI to build a RESTful API for managing student records.

Instructions:

- Endpoints required:
 - o GET /students → List all students
 - o POST /students → Add a new student
 - o PUT /students/{id} → Update student details
 - o DELETE /students/{id} → Delete a student record
- Use an in-memory data structure (list or dictionary) to store records.
- Ensure API responses are in JSON format

Prompt:

Generate a RESTful API using Python Flask for managing student records.

The API should support CRUD operations:

- GET /students → retrieve all students
- POST /students → add a new student
- PUT /students/{id} → update student details
- DELETE /students/{id} → delete a student

Use an in-memory dictionary or list to store data.

Ensure all responses are in JSON format.

Add proper error handling and comments.

Python Code:

```
1  #Generate a RESTful API using Python Flask for managing student records.
2  #The API should support CRUD operations:
3  #- GET /students → retrieve all students
4  #- POST /students → add a new student
5  #- PUT /students/{id} → update student details
6  #- DELETE /students/{id} → delete a student
7  #Use an in-memory dictionary or list to store data.Ensure all responses are in JSON format.Add proper error handling and comments.
8  from flask import Flask, request, jsonify
9
10 app = Flask(__name__)
11
12 # In-memory storage for students
13 students = {}
14
15
16 @app.route('/students', methods=['GET'])
17 def get_students():
18     """Retrieve all students."""
19     return jsonify({
20         "success": True,
21         "data": students
22     }), 200
23
24
25 @app.route('/students', methods=['POST'])
26 def add_student():
27     """Add a new student."""
28     try:
29         data = request.get_json()
```

```
try:
    data = request.get_json()

    # Validate input
    if not data:
        return jsonify({"error": "Request body must be JSON"}), 400

    student_id = data.get('id')
    if not student_id:
        return jsonify({"error": "Student ID is required"}), 400

    if student_id in students:
        return jsonify({"error": "Student already exists"}), 409

    # Create new student
    students[student_id] = {
        "name": data.get('name'),
        "age": data.get('age'),
        "grade": data.get('grade')
    }

    return jsonify({
        "message": "Student added successfully",
        "student": students[student_id]
    }), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500
```

```

1 @app.route('/students/<student_id>', methods=['PUT'])
2 def update_student(student_id):
3     """Update student details."""
4     try:
5         if student_id not in students:
6             return jsonify({"error": "Student not found"}), 404
7
8         data = request.get_json()
9         if not data:
10            return jsonify({"error": "Request body must be JSON"}), 400
11
12        # Update only provided fields
13        students[student_id].update({
14            "name": data.get('name', students[student_id]['name']),
15            "age": data.get('age', students[student_id]['age']),
16            "grade": data.get('grade', students[student_id]['grade'])
17        })
18
19        return jsonify({
20            "message": "Student updated successfully",
21            "student": students[student_id]
22        }), 200
23
24    except Exception as e:
25        return jsonify({"error": str(e)}), 500
26
27
28 @app.route('/students/<student_id>', methods=['DELETE'])

```

```

@app.route('/students/<student_id>', methods=['DELETE'])
def delete_student(student_id):
    """Delete a student."""
    try:
        if student_id not in students:
            return jsonify({"error": "Student not found"}), 404

        deleted_student = students.pop(student_id)

        return jsonify({
            "message": "Student deleted successfully",
            "student": deleted_student
        }), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# Run the Flask app
if __name__ == '__main__':
    app.run(debug=True)

```

REST API Methods Documentation:

The screenshot displays the Thunder Client interface. On the left, the 'Activity' panel lists several requests, with the most recent being a POST to 'localhost:5000/students' at 'just now'. The main panel shows a POST request to 'http://127.0.0.1:5000/students'. The 'Body' tab is selected, showing a JSON payload:

```
{ "id": "104", "name": "Siri", "age": 19, "grade": "A" }
```

. The right panel shows the response status as '201 CREATED' with a size of '118 Bytes' and a time of '21 ms'. The 'Response' tab is selected, displaying the JSON response:

```
{ "message": "Student added successfully", "student": { "age": 19, "grade": "A", "name": "Siri" } }
```

.

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

PUT 127.0.0.1:5000/students just now

POST localhost:3000/orders 24 days ago

PUT localhost:3000/students/69983bc03123... 1 month ago

POST localhost:3000/students 1 month ago

PUT localhost:3000/students/2 1 month ago

POST localhost:5000/students 1 month ago

PUT http://127.0.0.1:5000/students/101 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id": "101",
3   "name": "Suresh",
4   "age": 20,
5   "grade": "C"
6 }
```

Status: 200 OK Size: 122 Bytes Time: 11 ms

Response Headers 5 Cookies Results Docs {}

```
1 {
2   "message": "Student updated successfully",
3   "student": {
4     "age": 20,
5     "grade": "C",
6     "name": "Suresh"
7   }
8 }
```

Response Chart

course.py program15.py TC New Request localhost:5000/students TC New Request BLACKBOX

GET http://127.0.0.1:5000/students Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id": "104",
3   "name": "Siri",
4   "age": 19,
5   "grade": "A"
6 }
```

Status: 200 OK Size: 353 Bytes Time: 9 ms

Response Headers 5 Cookies Results Docs {}

```
1 {
2   "data": {
3     "101": {
4       "age": 20,
5       "grade": "C",
6       "name": "Suresh"
7     },
8     "102": {
9       "age": 21,
10      "grade": "B",
11      "name": "Ram"
12    },
13    "103": {
14      "age": 20,
15      "grade": "A",
16      "name": "Sreeja"
17    },
18    "104": {
19      "age": 19,
20      "grade": "A",
21      "name": "Siri"
22    }
23  },
24  "success": true
25 }
```

Output:

Pretty-print ☐

```
{
  "data": {
    "101": {
      "age": 20,
      "grade": "C",
      "name": "Suresh"
    },
    "102": {
      "age": 21,
      "grade": "B",
      "name": "Ram"
    },
    "103": {
      "age": 20,
      "grade": "A",
      "name": "Sreeja"
    },
    "104": {
      "age": 19,
      "grade": "A",
      "name": "Siri"
    }
  },
  "success": true
}
```

DELETE

http://127.0.0.1:5000/students/104

Query

Headers ²

Auth

Body ¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

```
1 {
2   "id": "104",
3   "name": "Siri",
4   "age": 19,
5   "grade": "A"
6 }
```

Status: 200 OK

Size: 120 Bytes

Time: 5 ms

Response

Headers ⁵

Cookies

Results

Docs

```
1 {
2   "message": "Student deleted successfully",
3   "student": {
4     "age": 19,
5     "grade": "A",
6     "name": "Siri"
7   }
8 }
```

After Deletion Output:

Pretty-print ☐

```
{
  "data": {
    "101": {
      "age": 20,
      "grade": "C",
      "name": "Suresh"
    },
    "102": {
      "age": 21,
      "grade": "B",
      "name": "Ram"
    },
    "103": {
      "age": 20,
      "grade": "A",
      "name": "Sreeja"
    }
  },
  "success": true
}
```

Justification:

1. Provides a CRUD interface to manage student information efficiently.
2. Uses in-memory storage (list/dictionary) for quick prototyping.
3. JSON responses ensure compatibility with web or mobile clients.
4. RESTful endpoints allow easy integration with front-end apps.
5. AI assists in generating code, test cases, and validation logic.

Task 2 : Library Book Management API

Task:

Develop a RESTful API to handle library books.

Instructions:

- Endpoints required:

- o GET /books → Retrieve all books
- o POST /books → Add a new book
- o GET /books/{id} → Get details of a specific book
 - o PATCH /books/{id} → Update partial book details
- (e.g., availability)
- o DELETE /books/{id} → Remove a book •

Implement error handling for invalid requests.

Prompt:

Generate a RESTful API using Python Flask for a Library Book Management system.

Requirements:

- GET /books → retrieve all books
- POST /books → add a new book
- GET /books/{id} → get book details by ID
- PATCH /books/{id} → partially update book details (like availability)
- DELETE /books/{id} → delete a book

Use an in-memory dictionary to store books.

Add proper error handling for invalid requests.

Return all responses in JSON format.

Python Code:

p15_2.py > update_book

```
1  #Generate a RESTful API using Python Flask for a Library Book Management system.
2  #Requirements:
3  #- GET /books → retrieve all books
4  #- POST /books → add a new book
5  #- GET /books/{id} → get book details by ID
6  #- PATCH /books/{id} → partially update book details (like availability)
7  #- DELETE /books/{id} → delete a book
8  #Use an in-memory dictionary to store books.
9  #Add proper error handling for invalid requests.
10 #Return all responses in JSON format.
11 from flask import Flask, request, jsonify
12 app = Flask(__name__)
13 # In-memory storage for books
14 books = {}
15 @app.route('/books', methods=['GET'])
16 def get_books():
17     """Retrieve all books."""
18     return jsonify({
19         "success": True,
20         "data": books
21     }), 200
22 @app.route('/books', methods=['POST'])
23 def add_book():
24     """Add a new book."""
25     try:
```

```
try:
    data = request.get_json()
    if not data:
        return jsonify({"error": "Request body must be JSON"}), 400
    book_id = data.get('id')
    if not book_id:
        return jsonify({"error": "Book ID is required"}), 400
    if book_id in books:
        return jsonify({"error": "Book already exists"}), 409
    books[book_id] = {
        "title": data.get('title'),
        "author": data.get('author'),
        "available": data.get('available', True)
    }
    return jsonify({
        "message": "Book added successfully",
        "book": books[book_id]
    }), 201
except Exception as e:
    return jsonify({"error": str(e)}), 500
@app.route('/books/<book_id>', methods=['GET'])
def get_book(book_id):
    """Get book details by ID."""
    book = books.get(book_id)
```

```

def get_book(book_id):
    if not book:
        return jsonify({"error": "Book not found"}), 404
    return jsonify({
        "success": True,
        "data": book
    }), 200

@app.route('/books/<book_id>', methods=['PATCH'])
def update_book(book_id):
    """Partially update book details."""
    book = books.get(book_id)
    if not book:
        return jsonify({"error": "Book not found"}), 404
    try:
        data = request.get_json()
        if not data:
            return jsonify({"error": "Request body must be JSON"}), 400
        # Update only provided fields
        for key in ['title', 'author', 'available']:
            if key in data:
                book[key] = data[key]
        return jsonify({
            "message": "Book updated successfully",
            "book": book
        }), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500

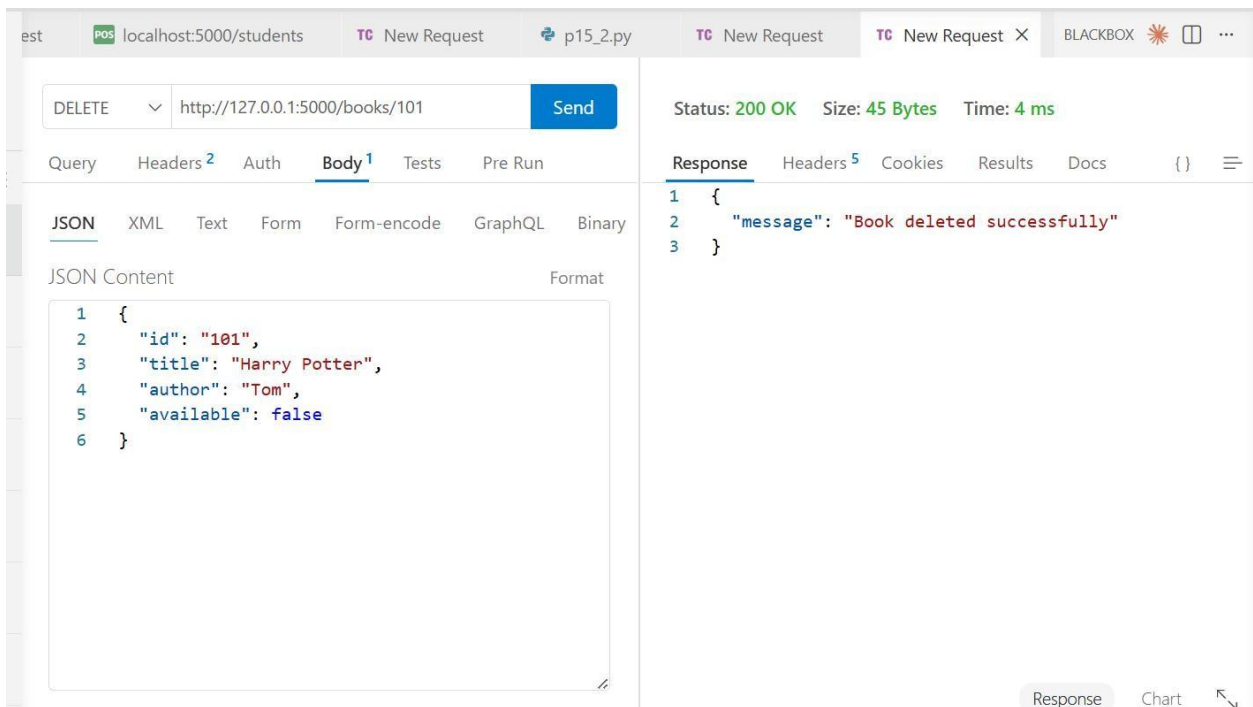
@app.route('/books/<book_id>', methods=['DELETE'])
def delete_book(book_id):
    """Delete a book."""
    if book_id not in books:
        return jsonify({"error": "Book not found"}), 404
    del books[book_id]
    return jsonify({
        "message": "Book deleted successfully"
    }), 200

if __name__ == '__main__':
    app.run(debug=True)

```

REST API Methods Documentation:


```
Pretty-print ☐
{
  "data": {
    "101": {
      "author": "J.K. Rowling",
      "available": true,
      "title": "Harry Potter"
    },
    "102": {
      "author": "John",
      "available": false,
      "title": "The route map of life"
    },
    "103": {
      "author": "Jerry",
      "available": false,
      "title": "Server Busy"
    }
  },
  "success": true
}
```



After Deletion Output:

Pretty-print ☐

```
{
  "data": {
    "102": {
      "author": "John",
      "available": false,
      "title": "The route map of life"
    },
    "103": {
      "author": "Jerry",
      "available": false,
      "title": "Server Busy"
    }
  },
  "success": true
}
```

Justification:

1. Enables management of library books with full CRUD support.
2. PATCH endpoint allows partial updates like book availability.
3. Error handling ensures robustness against invalid requests.
4. JSON responses standardize communication with other systems.
5. AI helps scaffold endpoints, suggest data validation, and create test cases.

Task 3 :Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

- Endpoints required:

o GET /employees → List all employees
o POST /employees → Add a new employee with salary details
o PUT /employees/{id}/salary → Update salary of an employee
o DELETE /employees/{id} → Remove employee from system

- Use AI to:

o Suggest data model structure.
o Add comments/docstrings for all endpoints.

Prompt:

Generate a RESTful API using Python Flask for managing employee payroll.

Requirements:

- GET /employees → list all employees
- POST /employees → add a new employee with salary
- PUT /employees/{id}/salary → update salary of an employee
- DELETE /employees/{id} → remove an employee

Use an in-memory dictionary to store employee data.

Add clear comments/docstrings for all endpoints.

Ensure all responses are in JSON format.

Python Code:

```

1  #Generate a RESTful API using Python Flask for managing employee payroll.
2  #Requirements:
3  #- GET /employees → list all employees
4  #- POST /employees → add a new employee with salary
5  #- PUT /employees/{id}/salary → update salary of an employee
6  #- DELETE /employees/{id} → remove an employee
7  #Use an in-memory dictionary to store employee data. Add clear comments/docstrings for all endpoints. Ensure all
8  from flask import Flask, request, jsonify
9  app = Flask(__name__)
10 # In-memory storage for employees
11 employees = {}
12 @app.route('/employees', methods=['GET'])
13 def get_employees():
14     """List all employees."""
15     return jsonify({
16         "success": True,
17         "data": employees
18     }), 200
19 @app.route('/employees', methods=['POST'])
20 def add_employee():
21     """Add a new employee with salary."""
22     try:
23         data = request.get_json()
24         if not data:
25
26             data = request.get_json()
27             if not data:
28                 return jsonify({"error": "Request body must be JSON"}), 400
29             employee_id = data.get('id')
30             if not employee_id:
31                 return jsonify({"error": "Employee ID is required"}), 400
32             if employee_id in employees:
33                 return jsonify({"error": "Employee already exists"}), 400
34             employees[employee_id] = {
35                 "name": data.get('name'),
36                 "salary": data.get('salary')
37             }
38             return jsonify({
39                 "message": "Employee added successfully",
40                 "employee": employees[employee_id]
41             }), 201
42     except Exception as e:
43         return jsonify({"error": str(e)}), 500
44 @app.route('/employees/<employee_id>/salary', methods=['PUT'])
45 def update_salary(employee_id):
46     """Update salary of an employee."""
47     employee = employees.get(employee_id)
48     if not employee:

```

```

def update_salary(employee_id):
    if not employee:
        return jsonify({"error": "Employee not found"}), 404
    try:
        data = request.get_json()
        if not data or 'salary' not in data:
            return jsonify({"error": "Salary is required in JSON body"}), 400
        employee['salary'] = data['salary']
        return jsonify({
            "message": "Salary updated successfully",
            "employee": employee
        }), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
@app.route('/employees/<employee_id>', methods=['DELETE'])
def delete_employee(employee_id):
    """Remove an employee."""
    if employee_id not in employees:
        return jsonify({"error": "Employee not found"}), 404
    del employees[employee_id]
    return jsonify({
        "message": "Employee removed successfully"
    }), 200
if __name__ == '__main__':

```

```

def delete_employee(employee_id):
    if employee_id not in employees:
        return jsonify({"error": "Employee not found"}), 404
    del employees[employee_id]
    return jsonify({
        "message": "Employee removed successfully"
    }), 200
if __name__ == '__main__':
    app.run(debug=True)

```

REST API Methods Documentation:

localhost:5000/students TC New Request p15_2.py p15_3.py TC New Request X TC BLACKBOX

POST http://127.0.0.1:5000/employees Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id": "1002",
3   "name": "Sreeja",
4   "salary": 100000
5 }
```

Status: 201 CREATED Size: 111 Bytes Time: 9 ms

Response Headers 5 Cookies Results Docs {}

```
1 {
2   "employee": {
3     "name": "Sreeja",
4     "salary": 100000
5   },
6   "message": "Employee added successfully"
7 }
```

PROBLEMS 28 OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE Python + - [] [] ... | ? X

localhost:5000/students TC New Request p15_2.py p15_3.py TC New Request X TC BLACKBOX

POST http://127.0.0.1:5000/employees Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id": "1003",
3   "name": "Satwika",
4   "salary": 1000000
5 }
```

Status: 201 CREATED Size: 113 Bytes Time: 10 ms

Response Headers 5 Cookies Results Docs {}

```
1 {
2   "employee": {
3     "name": "Satwika",
4     "salary": 1000000
5   },
6   "message": "Employee added successfully"
7 }
```

Response Chart ↗

GET

http://127.0.0.1:5000/employees

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

```
1 {
2   "id": "1003",
3   "name": "Satwika",
4   "salary": 1000000
5 }
```

Status: 200 OK

Size: 240 Bytes

Time: 8 ms

Response

Headers⁵

Cookies

Results

Docs

{ }

```
1 {
2   "data": {
3     "101": {
4       "name": "John",
5       "salary": 50000
6     },
7     "1002": {
8       "name": "Sreeja",
9       "salary": 100000
10    },
11    "1003": {
12      "name": "Satwika",
13      "salary": 1000000
14    }
15  },
16  "success": true
17 }
```

PUT

http://127.0.0.1:5000/employees/101/salary

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

```
1 {
2
3   "salary": 70000
4 }
```

Status: 200 OK

Size: 108 Bytes

Time: 10 ms

Response

Headers⁵

Cookies

Results

Docs

{ }

```
1 {
2   "employee": {
3     "name": "John",
4     "salary": 70000
5   },
6   "message": "Salary updated successfully"
7 }
```

Pretty-print ☐

```
{
  "data": {
    "1002": {
      "name": "Sreeja",
      "salary": 100000
    },
    "1003": {
      "name": "Satwika",
      "salary": 1000000
    },
    "101": {
      "name": "John",
      "salary": 70000
    }
  },
  "success": true
}
```

DELETE

▼

http://127.0.0.1:5000/employees/101

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

{

2

3

"salary": 70000

4

}

After Delection Output:

Pretty-print ☐

```
{
  "data": {
    "1002": {
      "name": "Sreeja",
      "salary": 100000
    },
    "1003": {
      "name": "Satwika",
      "salary": 1000000
    }
  },
  "success": true
}
```

Justification:

1. Manages employee records and salary updates via API.
2. Data model includes fields like id, name, department, salary.
3. PUT endpoint allows updating employee salary accurately.
4. JSON responses provide structured, auditable information.
5. AI generates docstrings, test cases, and validates data automatically.

Task 4 :Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

- Endpoints required:
 - o GET /menu → List available dishes
 - o POST /order → Place a new order
 - o GET /order/{id} → Track order status
 - o PUT /order/{id} → Update an existing order (e.g., change items)
 - o DELETE /order/{id} → Cancel an order
- AI should generate:
 - o REST API code
 - o Suggested improvements (like authentication, pagination)

Expected Output:

- Fully working API simulating a food ordering backend.

Prompt:

Generate a RESTful API using Python Flask for an Online Food Ordering System.

Requirements:

- GET /menu → list all available dishes
- POST /order → place a new order
- GET /order/{id} → get order status
- PUT /order/{id} → update order (change items)
- DELETE /order/{id} → cancel an order

Use in-memory data structures (dictionary/list).

Return all responses in JSON format.

Include proper error handling.

Also suggest improvements like authentication and pagination.

Python Code:

```
1 #Generate a RESTful API using Python Flask for an Online Food Ordering System.
2
3 #Requirements:
4 #- GET /menu → list all available dishes
5 #- POST /order → place a new order
6 #- GET /order/{id} → get order status
7 #- PUT /order/{id} → update order (change items)
8 #- DELETE /order/{id} → cancel an order
9
10 #Use in-memory data structures (dictionary/list).Return all responses in JSON format.Include proper error handling.Also suggest improvement
11 from flask import Flask, request, jsonify
12
13 app = Flask(__name__)
14
15 # Sample menu
16 menu = [
17     {"id": 1, "name": "Pizza", "price": 250},
18     {"id": 2, "name": "Burger", "price": 150},
19     {"id": 3, "name": "Pasta", "price": 200}
20 ]
21
22 # Orders storage
23 orders = {}
24 order_id = 1
25
26 # GET MENU
27 @app.route('/menu', methods=['GET'])
28 def get_menu():
29     return jsonify(menu)
```

```

5 # ✓ GET MENU
6
7 @app.route('/menu', methods=['GET'])
8 def get_menu():
9     return jsonify(menu)
10
11 # ✓ PLACE ORDER
12
13 @app.route('/order', methods=['POST'])
14 def place_order():
15     global order_id
16     data = request.json
17
18     if not data or 'items' not in data:
19         return jsonify({"error": "Items required"}), 400
20
21     order = {
22         "id": order_id,
23         "items": data["items"],
24         "status": "Placed"
25     }
26
27     orders[order_id] = order
28     order_id += 1
29
30     return jsonify(order), 201
31
32 # ✓ GET ORDER
33
34 @app.route('/order/<int:id>', methods=['GET'])
35 def get_order(id):
36     if id not in orders:

```

```

51 # ✅ GET ORDER
52 @app.route('/order/<int:id>', methods=['GET'])
53 def get_order(id):
54     if id not in orders:
55         return jsonify({"error": "Order not found"}), 404
56
57     return jsonify(orders[id])
58
59 # ✅ UPDATE ORDER
60 @app.route('/order/<int:id>', methods=['PUT'])
61 def update_order(id):
62     if id not in orders:
63         return jsonify({"error": "Order not found"}), 404
64
65     data = request.json
66
67     if 'items' in data:
68         orders[id]['items'] = data['items']
69
70     return jsonify(orders[id])
71
72 # ✅ DELETE ORDER
73 @app.route('/order/<int:id>', methods=['DELETE'])
74 def delete_order(id):
75     if id not in orders:
76         return jsonify({"error": "Order not found"}), 404
77
78     orders[id]['status'] = "Cancelled"
79     return jsonify({"message": "Order cancelled"})

```

```

69     return jsonify(orders[id])
70
71
72 # ✅ DELETE ORDER
73 @app.route('/order/<int:id>', methods=['DELETE'])
74 def delete_order(id):
75     if id not in orders:
76         return jsonify({"error": "Order not found"}), 404
77
78     orders[id]['status'] = "Cancelled"
79     return jsonify({"message": "Order cancelled"})
80
81 # ✅ RUN SERVER
82 if __name__ == '__main__':
83     app.run(debug=True)

```

REST API Methods Documentation:

request TC New Request TC New Request TC New Request p4.py TC New Request X POS loc BLACKBOX

POST http://127.0.0.1:5000/order Send

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "items": ["Biryani", "Chapathi"]
3 }
```

Status: 201 CREATED Size: 84 Bytes Time: 4 ms

Response Headers⁵ Cookies Results Docs {} ≡

```
1 {
2   "id": 2,
3   "items": [
4     "Biryani",
5     "Chapathi"
6   ],
7   "status": "Placed"
8 }
```

request TC New Request TC New Request TC New Request p4.py TC New Request X POS loc BLACKBOX

GET http://127.0.0.1:5000/menu Send

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "items": ["Biryani", "Chapathi"]
3 }
```

Status: 200 OK Size: 184 Bytes Time: 8 ms

Response Headers⁵ Cookies Results Docs {} ≡

```
1 [
2   {
3     "id": 1,
4     "name": "Pizza",
5     "price": 250
6   },
7   {
8     "id": 2,
9     "name": "Burger",
10    "price": 150
11  },
12  {
13    "id": 3,
14    "name": "Pasta",
15    "price": 200
16  }
17 ]
```

Update is ready, click to restart

Go Live Ready BLACKBOX AI Open Chat Go Live

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

GET 127.0.0.1:5000/order just now

DEL 127.0.0.1:5000/employees 3 hours ago

POST 127.0.0.1:5000/employees 3 hours ago

PUT 127.0.0.1:5000/books 3 hours ago

DEL 127.0.0.1:5000/students 5 hours ago

POST localhost:3000/orders 7 mins ago

PUT localhost:3000/students/69983bc0312... 1 month ago

POST localhost:3000/students 1 month ago

PUT localhost:3000/students/2

GET http://127.0.0.1:5000/order/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "items": ["Pizza", "Burger"]
3 }
```

Status: 200 OK Size: 84 Bytes Time: 4 ms

Response Headers 5 Cookies Results Docs {}

```
1 {
2   "id": 1,
3   "items": [
4     "Biryani",
5     "Chapathi"
6   ],
7   "status": "Placed"
8 }
```

Response Chart

17 11 BLACKBOX Agent Open Website Update is ready, click to restart. Go Live Ready BLACKBOXAI: Open Chat Go Live

DELETE http://127.0.0.1:5000/order/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "items": ["Pizza", "Burger"]
3 }
```

Status: 200 OK Size: 35 Bytes Time: 11 ms

Response Headers 5 Cookies Results Docs {}

```
1 {
2   "message": "Order cancelled"
3 }
```

PUT 

http://127.0.0.1:5000/order/1

Send

Query

Headers ²

Auth

Body ¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1 {

2 "items": ["Tea", "Burger"]

3 }

Status: 200 OK Size: 81 Bytes Time: 20 ms

Response

Headers ⁵

Cookies

Results

Docs

{ }



1 {

2 "id": 1,

3 "items": [

4 "Tea",

5 "Burger"

6],

7 "status": "Cancelled"

8 }

Response

Chart



Output:

Pretty-print ☐

```
[
  {
    "id": 1,
    "name": "Pizza",
    "price": 250
  },
  {
    "id": 2,
    "name": "Burger",
    "price": 150
  },
  {
    "id": 3,
    "name": "Pasta",
    "price": 200
  }
]
```

Justification:

1. Simulates a real-time food ordering system with CRUD operations.
2. Endpoints handle menu listing, order placement, updates, and tracking.
3. JSON ensures seamless communication with front-end apps.
4. AI suggests improvements like authentication, pagination, and order tracking.
5. Demonstrates practical application of REST APIs in e-commerce scenarios.